

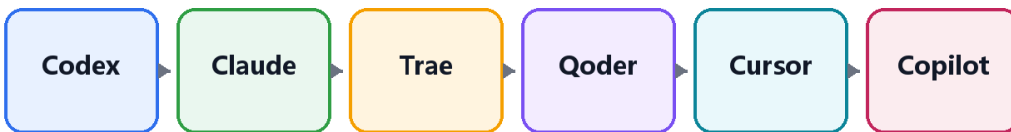
给 AI 编程工具装上一套项目纪律

面向开源社区和技术决策者的介绍

适读对象：开源贡献者、架构师、AI 工具使用者

多工具同一交付纪律

一个项目可以同时支持 Codex、Claude Code、Trae、Qoder、Cursor、GitHub Copilot 和 VS Code，但前提是它们



效率

少解释、少返工、快接手

风险

边界、验证、发布前置

知识

规则、ADR、Skills 沉淀

一个项目可以同时支持 Codex、Claude Code、Trae、Qoder、Cursor、GitHub Copilot 和 VS Code，但前提是它们读到同一套规则。

为什么需要工具无关的规则层

AI 编程工具变化很快，团队不可能把项目规范绑定在某一个工具里。今天用这个插件，明天换另一个 IDE，项目纪律不能跟着丢。

Enterprise AI Development OS

把规则、Skills、文档模板、审计脚本和适配器生成器放在仓库里，让不同工具从同一个源头获得约束。

这使得 AI

辅助开发从个人习惯变成项目资产：新人进入仓库、换工具、做交接时，都能从同一套入口开始。

公开版包含什么

- 规则入口：会话启动、开发顺序、目录边界、验证门禁。
- Skills：规划、架构、治理、前端、后端、数据、测试和部署能力单元。
- 文档记忆：Backlog、总控索引、模板、ADR 和回写结构。
- 审计门禁：方法论结构检查、开源边界检查、就绪度评分。

- 工具适配：把同一套规则投放到多种 AI 编程工具。

它不是什么

它不是通用 AI 应用平台，也不是完整的模型生命周期管理系统。它不承诺消除所有幻觉，也不声称当前已经实现完整的规则运行时。

它更像一层可迁移的工程治理骨架：先把项目规则、能力单元和验证方式组织好，再用审计与证据逐步增强。

如何参与或试用

可以先用 lite 模式安装到一个现有项目，只加入 AI

入口规则、文档模板和任务清单骨架。完整模式会复制官方 Skills、适配器工具和审计脚本。

贡献方向可以从规则改进、Skill 补充、适配器验证、公开案例脱敏和价值证据模板开始。所有公开贡献都应通过方法论审计、开源边界检查和适配器 dry-run。

“工具会变，项目纪律应该留在仓库里。”

建议行动

从 lite 模式开始，把一个项目的 AI 协作入口先统一起来。

公开口径边界

- 本文仅使用公开仓库中的规则、Skills、审计门禁、文档记忆和价值说明材料。
- 不包含私有策略、未脱敏案例、真实客户数据、本地路径或过程日志。
- 不宣称固定节省比例、固定缺陷降低比例或必然交付成功。